Lec5. Training Part III

How to train network

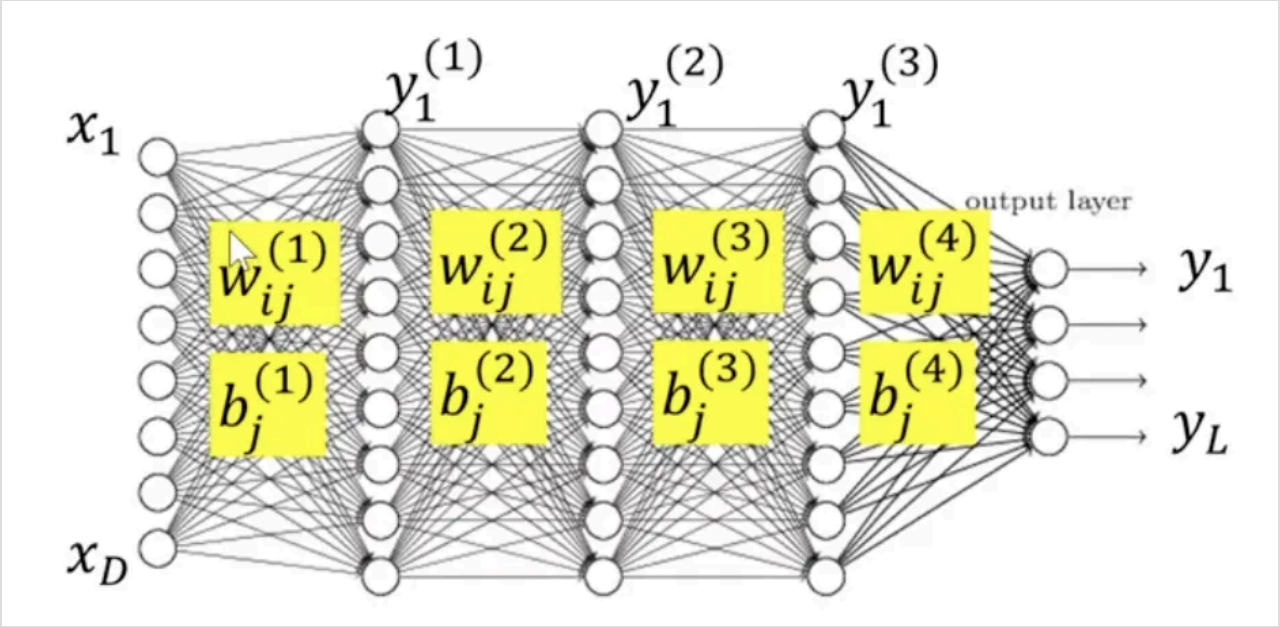
$$Loss(W) = \frac{1}{T}\sum_{i} div(Y_i, d_i)$$

- 학습 데이터가 T개의 샘플로 구성되어 있다고 하자. $Y_i = F_\theta(x_i)$는 i번째 입력에 대한 신경망 예측값이고, d_i는 정답이며, $\theta=\{w_{ij}^{(k)}, b_j^{(k)}\}$는 모든 학습 가능한 가중치와 편향의 집합이다. 표본별 손실함수를 ℓ로 정의하면 전체 평균 손실은 $L(\theta) = (1/T)\ \Sigma_{i=1}^{T}\ \ell(Y_i, d_i)$이다. 학습에서는 선택한 손실함수와 출력 활성화에 따라 gradient를 계산하고, $L(\theta)$를 감소시키도록 θ를 갱신한다.

$$\text{Minimize } Loss \text{ w.r.t } \left\{w_{ij}^{(k)}, b_j^{(k)}\right\} \text{ using gradient descent}$$

- 신경망 학습은 입력을 이용한 순전파, 손실 계산, 역전파를 통한 gradient 계산, 매개변수 갱신의 순서로 수행된다. 학습률을 $\eta>0$라 하면 gradient descent의 업데이트는 $w_{ij}^{(k)} \leftarrow w_{ij}^{(k)} - \eta\ \partial L/\partial w_{ij}^{(k)}$, $b_j^{(k)} \leftarrow b_j^{(k)} - \eta\ \partial L/\partial b_j^{(k)}$이다. 미니배치 학습에서는 해당 미니배치의 평균 손실에 대한 gradient를 계산하여 같은 방식으로 갱신한다.

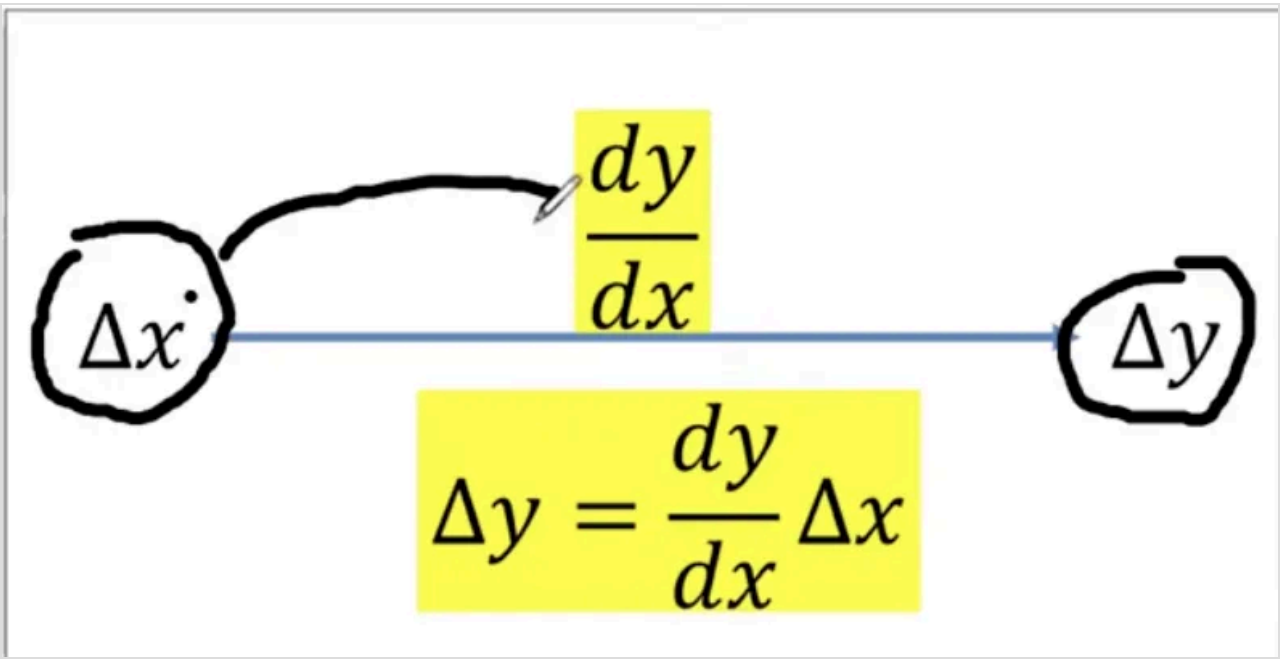
Notation



Influence diagram

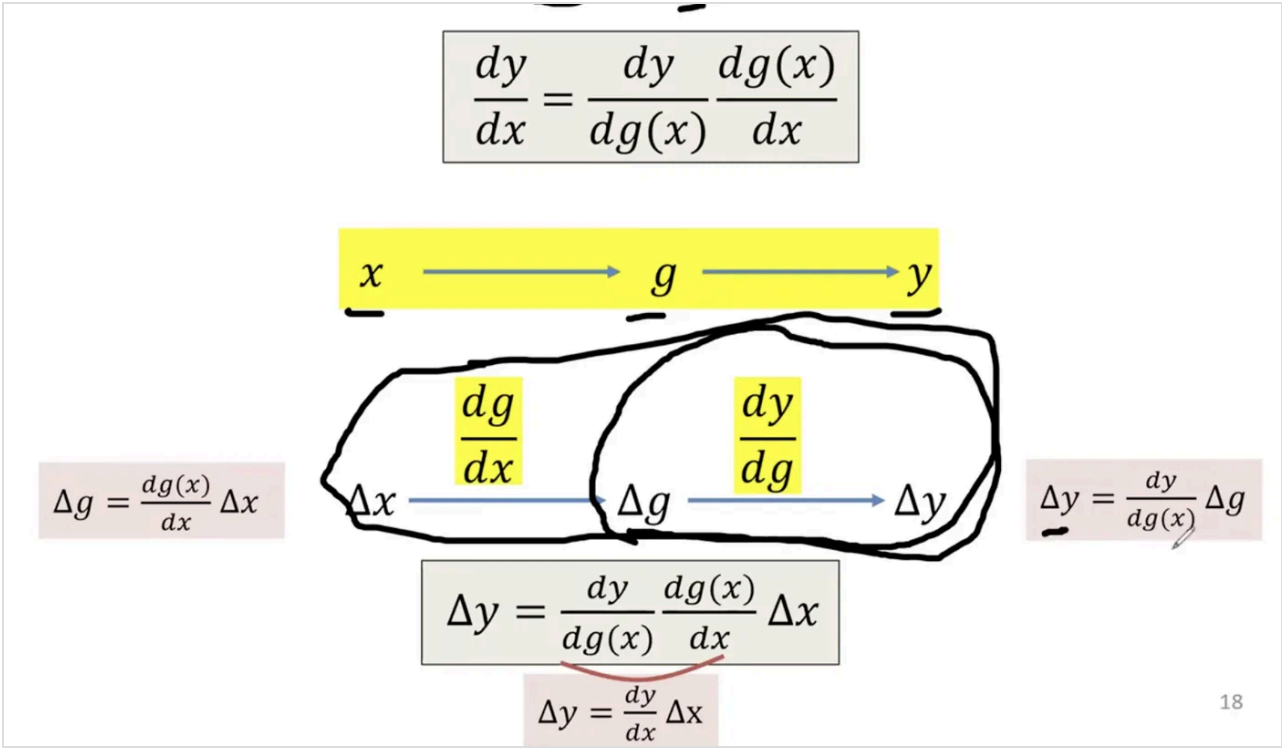


- Influence diagram에서 x→y는 y가 x에 의존하는 계산 관계를 나타낸다. 다만 x가 변한다고 해서 y가 반드시 변하는 것은 아니며, 특정 지점에서는 변화에 대한 민감도 dy/dx가 0일 수 있다. 미분 가능한 경우 dy/dx는 x의 미소 변화에 대한 y의 국소적인 변화율을 나타낸다.



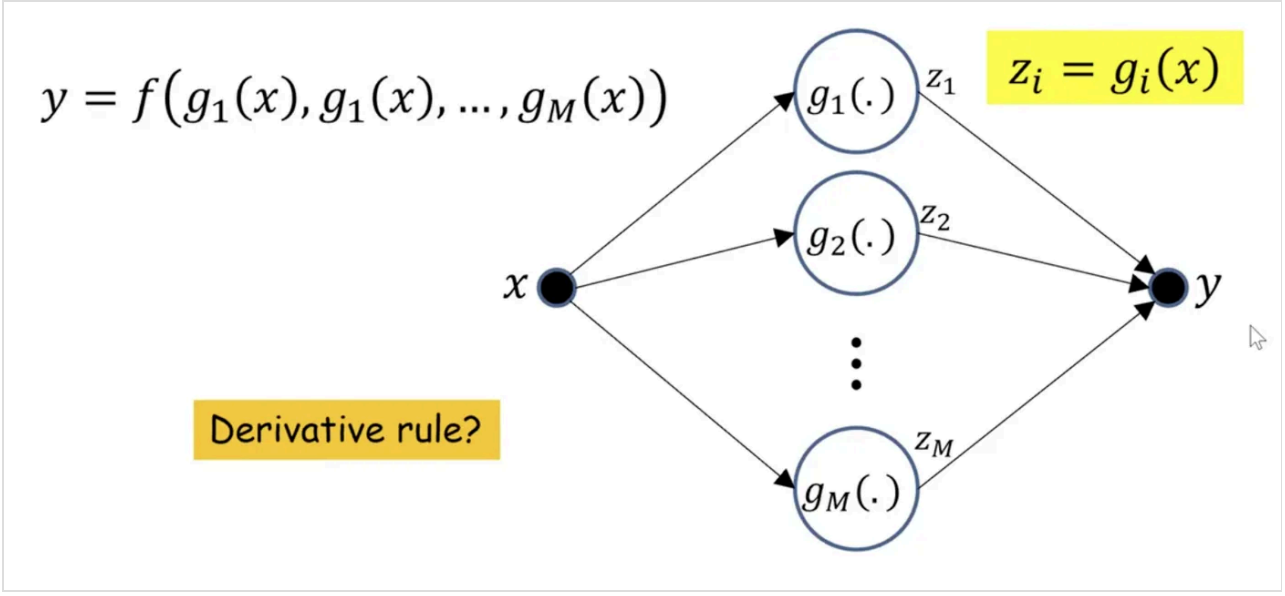
- x=x(θ)가 매개변수 θ에 의존하고 y=f(x)가 미분 가능하면, 연쇄법칙에 따라 $\nabla_\theta y = (dy/dx)\nabla_\theta x$가 성립한다.
  즉, 최종 출력의 매개변수에 대한 gradient는 중간 변수에 대한 국소 미분계수와 해당 중간 변수의 gradient를 곱하여 구한다.

Chain rule

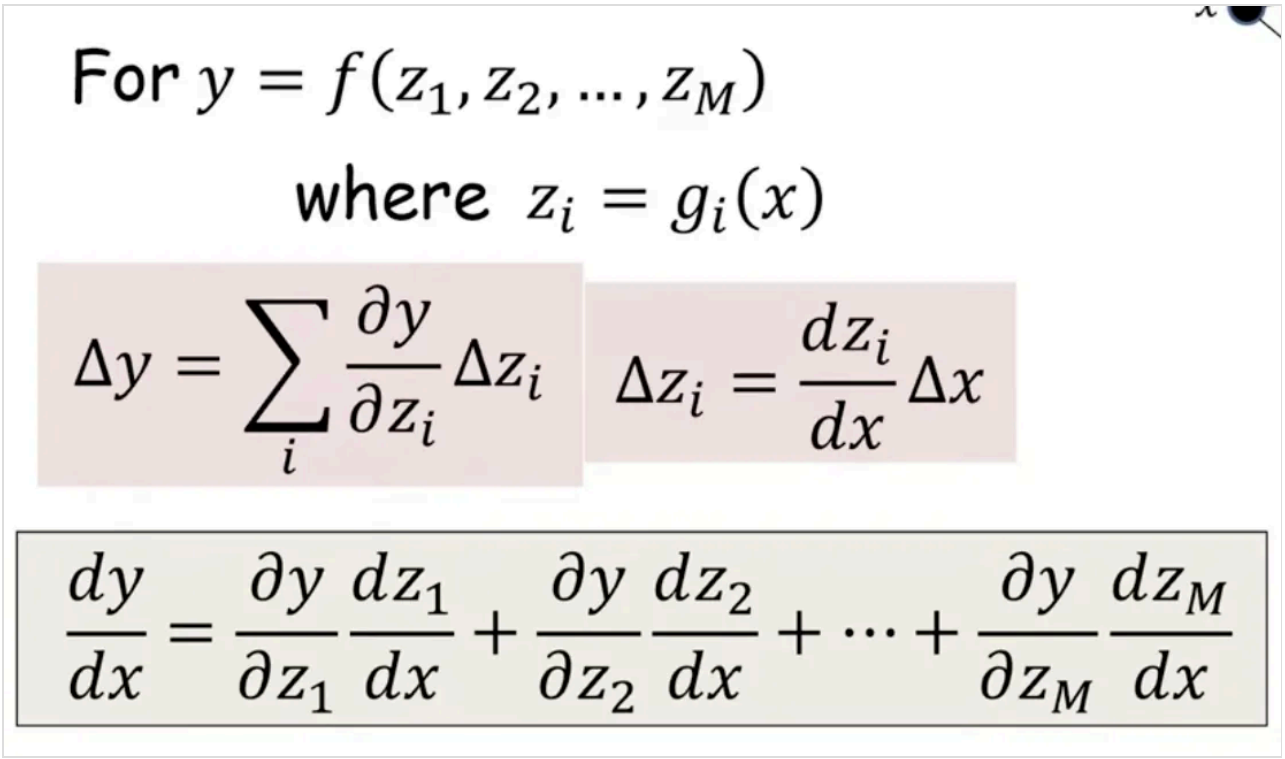


- x에 대한 y의 변화를 보기 위해서는,
  중간에 있는 ∇g를 이용한 **chain rule**을 보면 됨.

Distributed Chain Rule



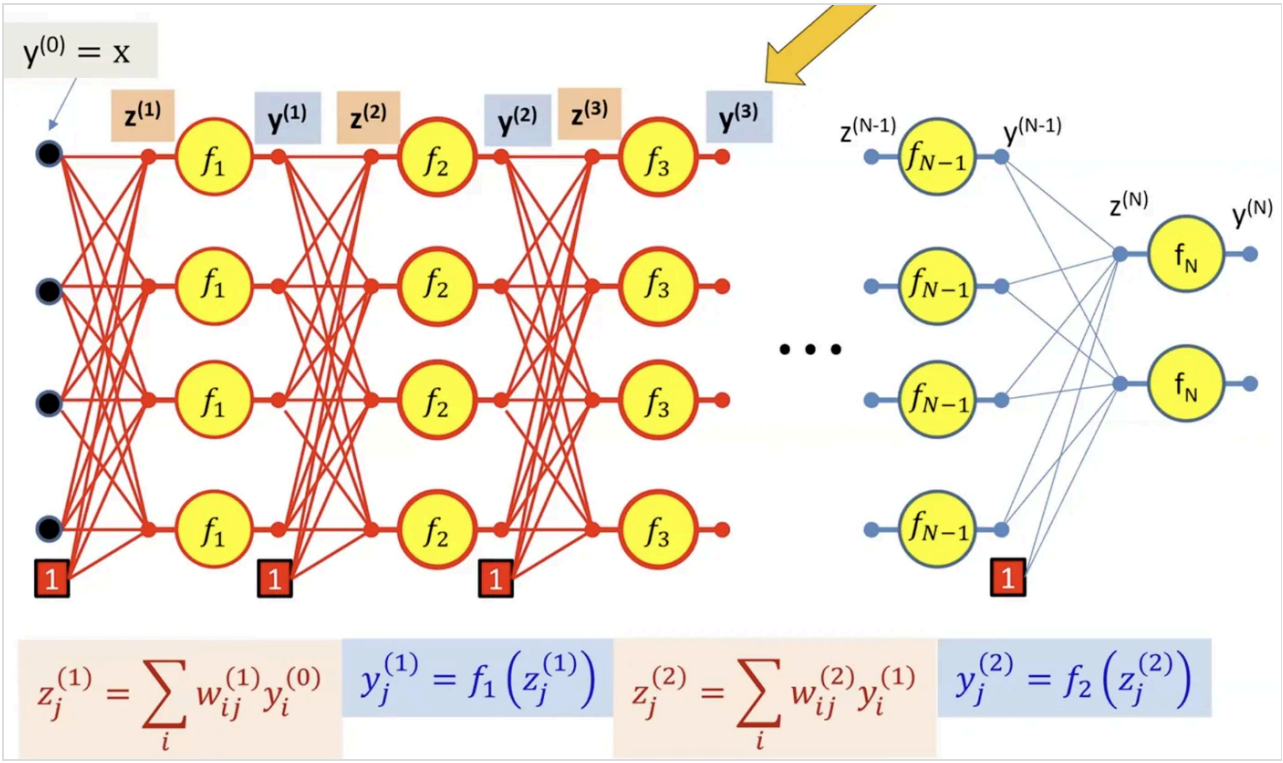
- 정정: 위 강의 캡처의 식에서 두 번째 $g_1(x)$는 $g_2(x)$의 오기입니다. 올바른 식은 $y = f(g_1(x), g_2(x), \cdots, g_M(x))$ 이며, 각 중간 변수를 $z_i = g_i(x)$로 정의했을 때 최종 합산식 $dy/dx = \Sigma_i (\partial y/\partial z_i)(dz_i/dx)$는 정확합니다.
- 이 경우, x가 변할 때 $g_1, g_2, \ldots$들이 변하므로, 각각에 대한 **편미분의 chain rule**을 구할 수 있어야 함. → 아래와 같이



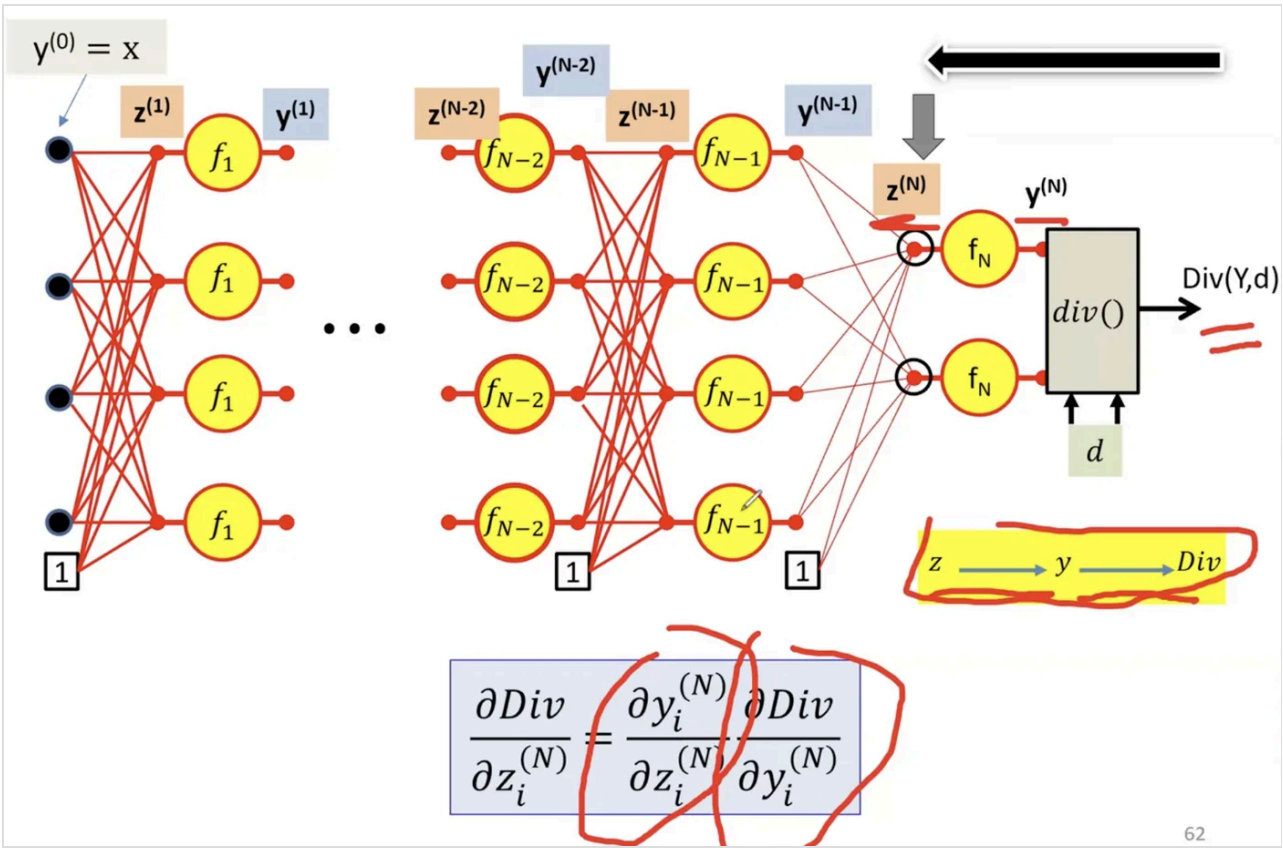
- 미분 가능한 함수 $y=f(x)$에서 미소 변화에 대한 정확한 미분 관계는 $dy=f'(x)dx$이다. 유한하지만 충분히 작은 변화량에 대해서는 $\Delta y \approx f'(x)\Delta x$라는 1차 근사를 사용할 수 있다. 마찬가지로 여러 변수에 의존하는 미분 가능한 함수 $y=f(z_1,\cdots,z_M)$에서는 $\Delta y \approx \Sigma_i (\partial y/\partial z_i)\Delta z_i$가 성립하며, 변화량이 0에 가까워질수록 무시한 고차항의 영향이 작아진다.

How to train network??

Forward path

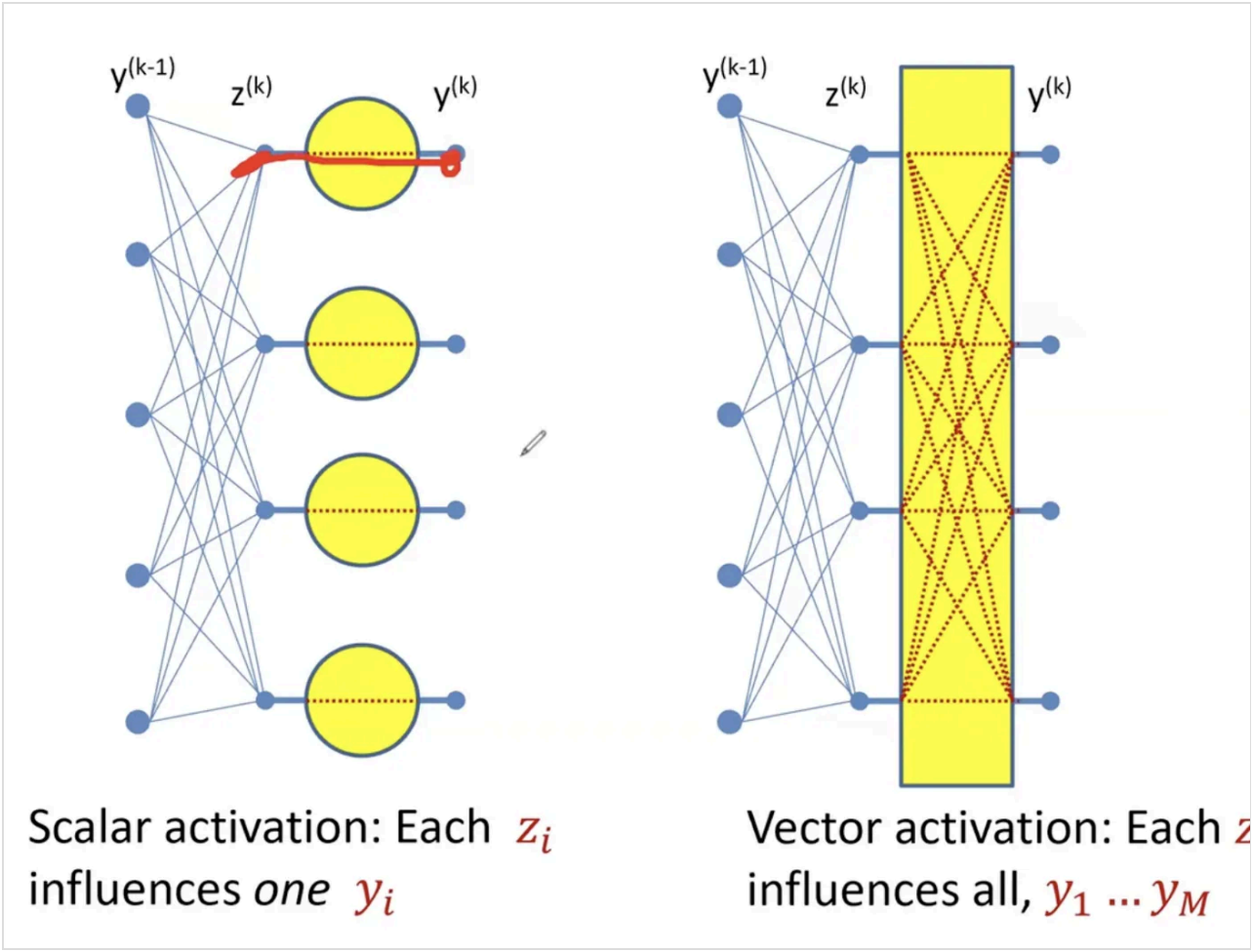


Backward path

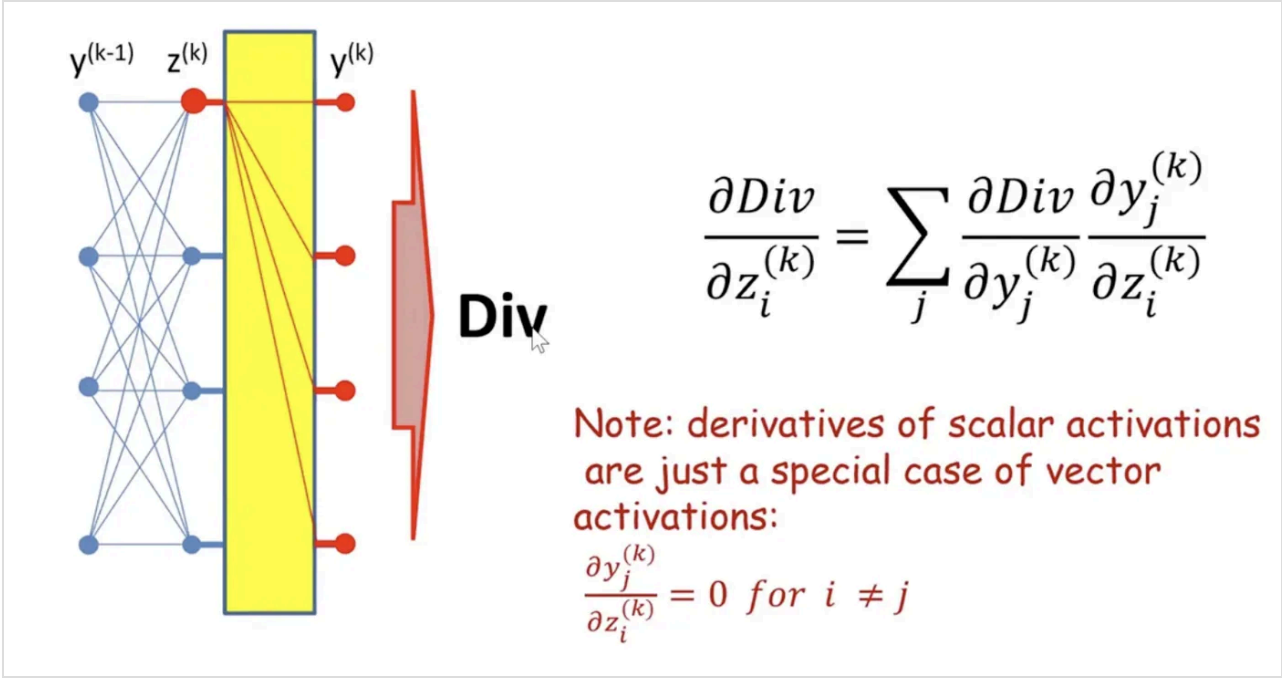


- 역전파에서는 각 층의 pre-activation $z_j^{(k)}$에 대한 손실의 미분값을 $\delta_j^{(k)}=\partial \ell/\partial z_j^{(k)}$로 정의한다. 성분별 활성화 $y_j^{(k)}=f_k(z_j^{(k)})$를 사용하는 신경망에서는 출력층에서 $\delta_j^{(N)}=(\partial \ell/\partial y_j^{(N)})f_N'(z_j^{(N)})$를 계산하고, 은닉층에서는 $\delta_i^{(k)}=f_k'(z_i^{(k)})\ \Sigma_j\ w_{ij}^{(k+1)}\ \delta_j^{(k+1)}$를 통해 뒤쪽 층의 gradient를 전달한다. 순전파가 $z_j^{(k)}=\Sigma_i\ w_{ij}^{(k)}\ y_i^{(k-1)}+b_j^{(k)}$라면 한 샘플의 gradient는 $\partial \ell/\partial w_{ij}^{(k)}=\delta_j^{(k)}\ y_i^{(k-1)}$, $\partial \ell/\partial b_j^{(k)}=\delta_j^{(k)}$이다. 전체 데이터셋의 평균 손실을 사용하는 경우에는 이 샘플별 gradient들을 평균한다.

Special Case 1. Vector activations

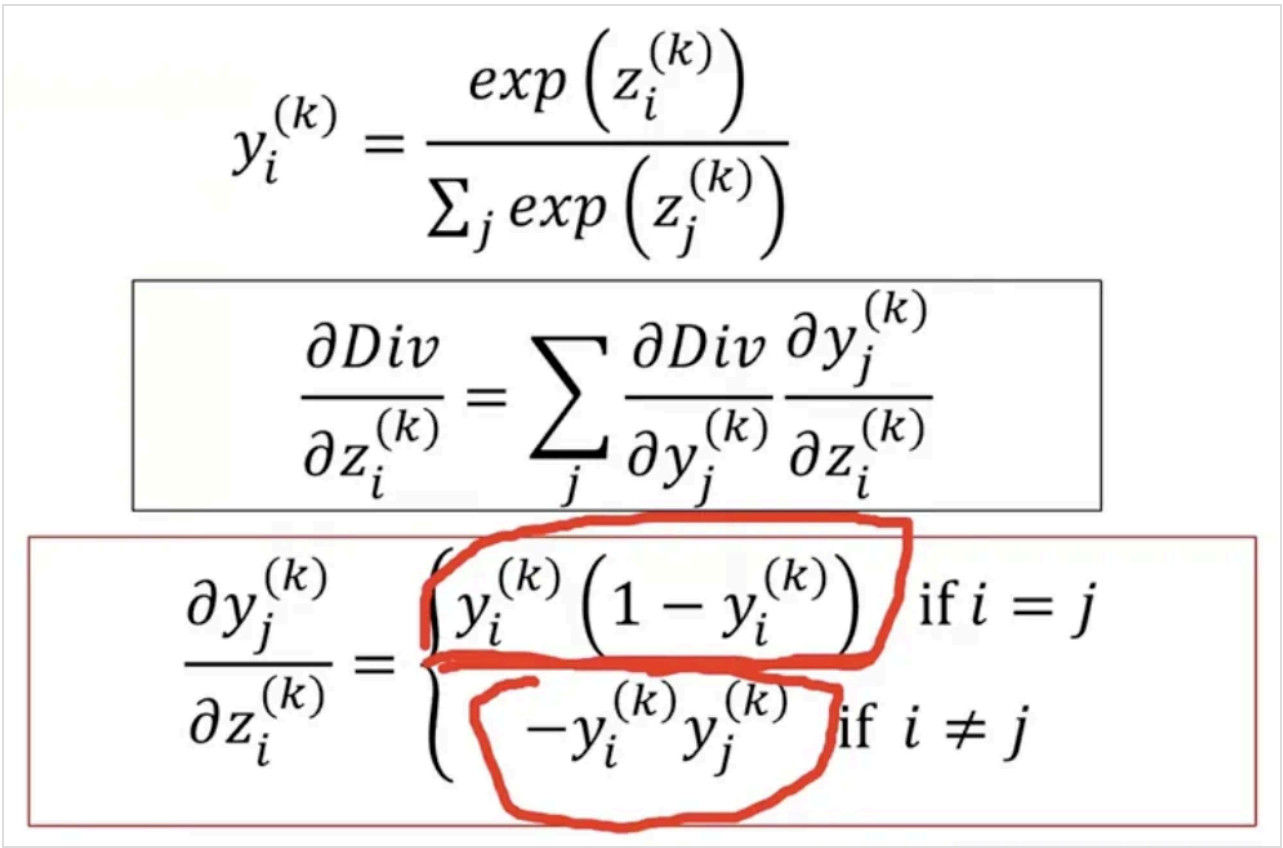


- Softmax처럼 출력 성분들이 서로 결합된 벡터 활성화 y=f(z)에서는 하나의 입력 z_i가 여러 출력 y_j에 영향을 줄 수 있다. 따라서 손실 L의 입력별 gradient는 $\partial L/\partial z_i=\Sigma_j\ (\partial L/\partial y_j)(\partial y_j/\partial z_i)$로 계산한다. 반면 성분별 활성화 함수에서는 $i\neq j$인 교차 편미분이 0이므로 합산식이 해당 성분의 항 하나로 단순화된다.



- 따라서, derivatives 구할 때, y변화에 대한 영향을 모두 더해야 함.

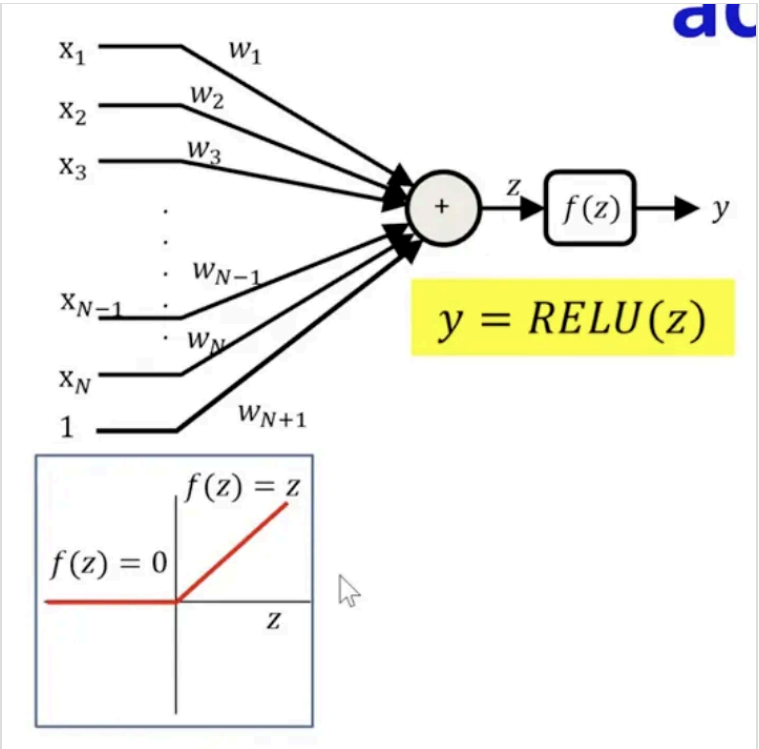
EX) Softmax

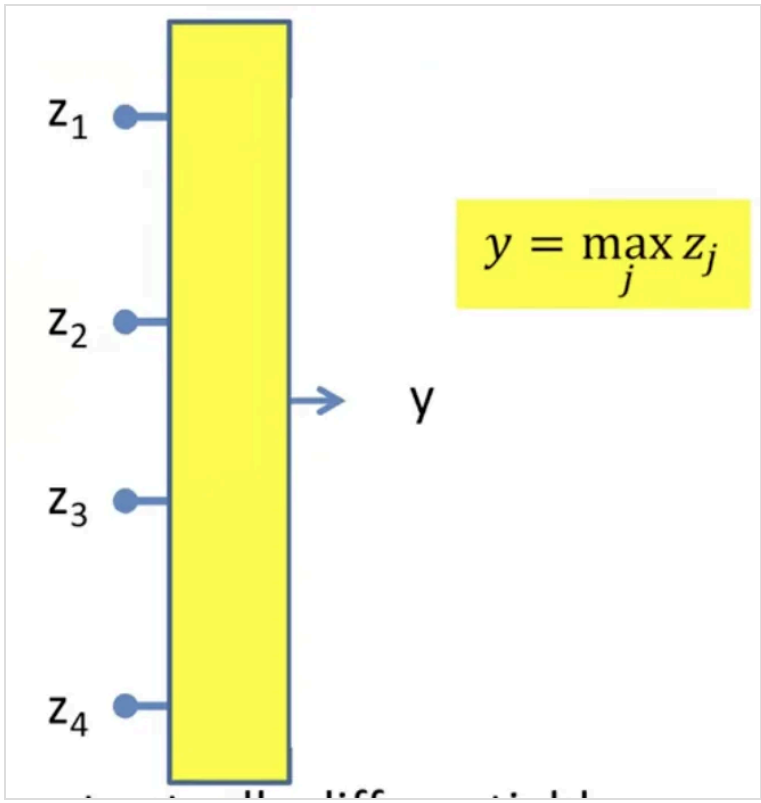


- Softmax를 $y_j = e^{z_j}/S$, $S=\Sigma_k e^{z_k}$로 정의하자. 분모는 모든 z_i에 의존하므로 $\partial S/\partial z_i = e^{z_i}$이다. 따라서 몫의 미분법칙을 적용하면 $\partial y_j/\partial z_i = (1_{i=j}\, e^{z_j}\, S - e^{z_j}\, e^{z_i})/S^2 = y_j(1_{i=j} - y_i)$가 된다. 여기서 $1_{i=j}$는 i=j이면 1, 그렇지 않으면 0이다. 결과적으로 대각 항은 $y_i(1-y_i)$, 비대각 항은 $-y_i\, y_j$가 되며, 이 편미분들을 이용해 역전파에서 각 입력 z_i에 대한 손실 gradient를 계산할 수 있다.

Special Case : Non-differentiable activations

ReLU(z)





- ReLU(z)=max(0,z)는 구간별 선형 함수로, z<0에서는 미분값이 0이고 z>0에서는 미분값이 1이다. z=0에서는 좌미분계수 0과 우미분계수 1이 달라 일반적인 미분이 존재하지 않는다. 다만 이 지점의 convex subgradient 집합은 [0,1]이므로 역전파 구현에서는 이 범위의 값을 선택할 수 있으며, 흔히 0을 사용한다.

$$\frac{\partial y}{\partial z_i} = \begin{cases} 1, & i = \underset{j}{\operatorname{argmax}}\, z_j \\ 0, & otherwise \end{cases}$$

- $y=\max_j z_j$에서 최댓값을 갖는 인덱스 j^*가 유일하면 $\partial y/\partial z_{j^*}=1$이고 나머지 성분의 편미분은 0이다. 반면 여러 성분이 동일한 최댓값을 갖는 동점 지점에서는 일반적인 gradient가 존재하지 않는다. 이 경우 최댓값을 갖는 성분들 중 하나에 1을 전달하는 one-hot 벡터를 subgradient로 선택할 수 있으며, 활성 최대 성분들에 대한 one-hot 벡터의 볼록결합도 subgradient가 된다.